

LangChain

Chat Models Basics

1. model.invoke command sends the prompt and gets a response back.

```
# Invoke the model with messages
result = model.invoke(messages)
print(f"Answer from AI: {result.content}")
```

2. System-Human-AI-Human-AI.... Conversations

```
# SystemMessage:
# Message for priming AI behavior, usually passed in as the first of a sequence
# HumanMessage:
# Message from a human to the AI model.
messages = [
    SystemMessage(content="Solve the following math problems"),
    HumanMessage(content="What is 81 divided by 9?"),
]
```

3. By creating and updating the list with the chat prompts and responses, we can pass the entire list to the Chatbot so it has full context of the previous conversation.

```
chat_history = [] # Use a list to store messages

# Set an initial system message (optional)
system_message = SystemMessage(content="You are a helpful AI assistant.")
chat_history.append(system_message) # Add system message to chat history

# Chat loop
while True:
    query = input("You: ")
```

```

if query.lower() == "exit":
    break
chat_history.append(HumanMessage(content=query)) # Add user message

# Get AI response using history
result = model.invoke(chat_history)
response = result.content
chat_history.append(AIMessage(content=response)) # Add AI message

print(f"AI: {response}")

print("----- Message History -----")
print(chat_history)

```

4. We can upload our chat to the cloud so that we can access it from anywhere anytime using Firebase Database. Login into google using "gcloud auth application-default login" from the command prompt.